HouseHold_Power_Consumption

October 9, 2026

1 Household Power Consumption

Autor: Andrés Aranguren
Fecha: Noviembre 2025
Dataset: Household Power Consumption - UCI —

1.1 Contexto del Proyecto

Este análisis examina el consumo de energía eléctrica en un hogar durante un período determinado.

1.1.1 Objetivos

1. **Analizar el consumo energético total del hogar**, incluyendo variaciones por minuto, hora y día.
2. **Identificar patrones temporales de consumo**, como horarios de mayor uso y posibles estacionalidades.
3. **Detectar cuáles dispositivos o áreas generan mayor gasto energético**, según submetering registrado.
4.

1.2 Generar recomendaciones accionables para optimizar el uso de energía y reducir consumo/ costos.

1.3 Variables del Dataset

Variable	Descripción
Date	Fecha del registro de consumo
Time	Hora del registro
Global_active_power	Potencia activa global (kW)
Global_reactive_power	Potencia reactiva global (kW)
Voltage	Voltaje promedio por minuto (V)
Global_intensity	Intensidad eléctrica promedio (A)
Sub_metering_1	Consumo cocina (lavavajillas, horno, microondas)
Sub_metering_2	Consumo lavandería (lavadora, secadora, nevera)
Sub_metering_3	Consumo aire acondicionado / calentador

1.4 1. Carga y Exploración Inicial

En esta sección cargaremos el dataset y realizaremos una exploración inicial para entender: - Estructura de los datos - Tipos de variables - Primeras y últimas filas - Distribución general de potencias y submeterings

[1]:
```
# Importar librerías necesarias
import pandas as pd
import numpy as np
from datetime import datetime
import warnings
# Visualizacion
import matplotlib.pyplot as plt
import seaborn as sns


warnings.filterwarnings('ignore')

# Configuración de visualización
sns.set_theme(style="darkgrid")
sns.set_palette("husl")
#%matplotlib inline

# Configurar pandas
pd.set_option('display.width', 2000)
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 120)

print(" Librerías importadas correctamente y configuración de graficos␣
 ↪aplicada.")
```

```
 Librerías importadas correctamente y configuración de graficos aplicada.
```

[2]:
```
df_house = pd.read_csv('../data/household_power_consumption.txt', sep=';',␣
 ↪low_memory=False, na_values='?')
df_house.replace('?', np.nan, inplace=True)

print("\n  INFORMACIÓN GENERAL DEL DATASET")
print("="*50)
print("El tamaño del dataset es:", df_house.shape, " (filas, columnas)")
print("\nTipos de datos por columna:")
print("="*50)
```

```
  INFORMACIÓN GENERAL DEL DATASET
==================================================
El tamaño del dataset es: (2075259, 9)  (filas, columnas)

Tipos de datos por columna:
```

```
==================================================
```

[3]:
```
print("="*50)
print("\nPrimeras 5 filas del dataset:")
print(df_house.head())
print("\nÚltimas 5 filas del dataset:")
print(df_house.tail())
print("="*50)
```

```
==================================================

Primeras 5 filas del dataset:
         Date      Time  Global_active_power  Global_reactive_power  Voltage
Global_intensity  Sub_metering_1  Sub_metering_2  Sub_metering_3
0  16/12/2006  17:24:00                4.216                  0.418   234.84
18.4              0.0             1.0             17.0
1  16/12/2006  17:25:00                5.360                  0.436   233.63
23.0              0.0             1.0             16.0
2  16/12/2006  17:26:00                5.374                  0.498   233.29
23.0              0.0             2.0             17.0
3  16/12/2006  17:27:00                5.388                  0.502   233.74
23.0              0.0             1.0             17.0
4  16/12/2006  17:28:00                3.666                  0.528   235.68
15.8              0.0             1.0             17.0

Últimas 5 filas del dataset:
               Date      Time  Global_active_power  Global_reactive_power
Voltage  Global_intensity  Sub_metering_1  Sub_metering_2  Sub_metering_3
2075254  26/11/2010  20:58:00                0.946                   0.0
240.43               4.0             0.0             0.0             0.0
2075255  26/11/2010  20:59:00                0.944                   0.0
240.00               4.0             0.0             0.0             0.0
2075256  26/11/2010  21:00:00                0.938                   0.0
239.82               3.8             0.0             0.0             0.0
2075257  26/11/2010  21:01:00                0.934                   0.0
239.70               3.8             0.0             0.0             0.0
2075258  26/11/2010  21:02:00                0.932                   0.0
239.55               3.8             0.0             0.0             0.0
==================================================
```

[4]:
```
print("="*50)
print("\nCantidad de valores nulos por columna:")
print(df_house.isnull().sum())
print("="*50)
```

```
==================================================

Cantidad de valores nulos por columna:
Date                      0
```

```
Time                          0
Global_active_power       25979
Global_reactive_power     25979
Voltage                   25979
Global_intensity          25979
Sub_metering_1            25979
Sub_metering_2            25979
Sub_metering_3            25979
dtype: int64
================================================
```

[5]:
```
# El tolist() convierte el índice de columnas en una lista normal
print(f"Nombres de las columnas: {df_house.columns.tolist()}")
#info muestra el tipo de dato de cada columna y si tiene valores nulos
print("\n  INFORMACIÓN DE COLUMNAS:")
df_house.info()
```

```
Nombres de las columnas: ['Date', 'Time', 'Global_active_power',
'Global_reactive_power', 'Voltage', 'Global_intensity', 'Sub_metering_1',
'Sub_metering_2', 'Sub_metering_3']

  INFORMACIÓN DE COLUMNAS:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2075259 entries, 0 to 2075258
Data columns (total 9 columns):
 #   Column                 Dtype
---  ------                 -----
 0   Date                   object
 1   Time                   object
 2   Global_active_power    float64
 3   Global_reactive_power  float64
 4   Voltage                float64
 5   Global_intensity       float64
 6   Sub_metering_1         float64
 7   Sub_metering_2         float64
 8   Sub_metering_3         float64
dtypes: float64(7), object(2)
memory usage: 142.5+ MB
```

[6]:
```
df_house.describe()
```

[6]:
```
       Global_active_power  Global_reactive_power       Voltage
Global_intensity  Sub_metering_1  Sub_metering_2  Sub_metering_3
count         2.049280e+06           2.049280e+06  2.049280e+06
2.049280e+06    2.049280e+06    2.049280e+06    2.049280e+06
mean          1.091615e+00           1.237145e-01  2.408399e+02
4.627759e+00    1.121923e+00    1.298520e+00    6.458447e+00
std           1.057294e+00           1.127220e-01  3.239987e+00
```

```
4.444396e+00    6.153031e+00    5.822026e+00    8.437154e+00
min           7.600000e-02          0.000000e+00  2.232000e+02
2.000000e-01    0.000000e+00    0.000000e+00    0.000000e+00
25%           3.080000e-01          4.800000e-02  2.389900e+02
1.400000e+00    0.000000e+00    0.000000e+00    0.000000e+00
50%           6.020000e-01          1.000000e-01  2.410100e+02
2.600000e+00    0.000000e+00    0.000000e+00    1.000000e+00
75%           1.528000e+00          1.940000e-01  2.428900e+02
6.400000e+00    0.000000e+00    1.000000e+00    1.700000e+01
max           1.112200e+01          1.390000e+00  2.541500e+02
4.840000e+01    8.800000e+01    8.000000e+01    3.100000e+01
```

1.4.1 Conclusiones iniciales

El dataset contiene 1,048,575 filas y 9 columnas, con un tamaño considerable.

Ocho columnas numéricas aparecen como object, lo que indica que deben convertirse a valores numéricos para permitir análisis.

Hay 4,069 valores faltantes en casi todas las columnas numéricas, probablemente por valores inválidos en el archivo original.

Sub_metering_3 es la única columna ya en formato numérico (float64) y muestra una distribución coherente.

No hay columnas vacías, pero sí problemas de formato y consistencia.

Conclusión general

Los datos no están listos para análisis: requieren conversión de tipos, manejo de valores faltantes y creación de una columna Datetime.

1.5 2. Limpieza y Preparación

En esta sección dejaremos los datos en un formato adecuado para el análisis.
Realizaremos tareas clave de preprocesamiento para asegurar coherencia, calidad y correcta estructura temporal del dataset.

1.5.1 En esta etapa haremos:

- **Unir `Date` y `Time` en una sola columna `Datetime`** para trabajar con series temporales.
- **Convertir las columnas numéricas** (`Global_active_power`, `Voltage`, `Global_intensity`, `Sub_metering_*`, etc.) desde texto (`object`) a valores numéricos.
- **Identificar y tratar valores nulos**, que representan aproximadamente **un porcentaje** del dataset, el cual encontraremos.
- **Crear la columna de consumo no medido** por los sub-meterings para conocer la energía restante no asignada a áreas específicas de la casa.

Estas transformaciones son esenciales para obtener un dataset limpio y listo para análisis estadístico y visualizaciones posteriores.

[7]:
```
# 1. Crear directamente la columna Datetime desde los valores de texto␣
 ↪originales
df_house['Datetime'] = pd.to_datetime(
    df_house['Date'].astype(str) + ' ' + df_house['Time'].astype(str),
    format='%d/%m/%Y %H:%M:%S',
    errors='coerce'
)

# 2. Extraer Date y Time directamente desde Datetime (ya no necesitas parsear␣
 ↪por separado)
df_house['Date'] = df_house['Datetime'].dt.date
df_house['Time'] = df_house['Datetime'].dt.time

print("\n  Columna 'Datetime' creada correctamente.")

# 3. Convertir columnas numéricas
cols_numericas = [
    "Global_active_power", "Global_reactive_power",
    "Voltage", "Global_intensity",
    "Sub_metering_1", "Sub_metering_2", "Sub_metering_3"
]
df_house[cols_numericas] = df_house[cols_numericas].apply(pd.to_numeric,␣
 ↪errors='coerce')

print("\n  Conversión de columnas numéricas completada.")

df_house.dtypes
```

```
 Columna 'Datetime' creada correctamente.

 Conversión de columnas numéricas completada.
```

[7]:
```
Date                          object
Time                          object
Global_active_power          float64
Global_reactive_power        float64
Voltage                      float64
Global_intensity             float64
Sub_metering_1               float64
Sub_metering_2               float64
Sub_metering_3               float64
Datetime              datetime64[ns]
dtype: object
```

[8]:
```
print("\n  Se mostrará el consumo de energía en el dataset:")
```

```
consumo_min = df_house["Global_active_power"].idxmin()
consumo_max = df_house["Global_active_power"].idxmax()

def resumen(index):
    return (
        f"Consumo: {df_house.loc[index,'Global_active_power']} kW\n"
        f"Voltaje: {df_house.loc[index,'Voltage']} V\n"
        f"Intensidad: {df_house.loc[index,'Global_intensity']} A\n"
        f"Fecha y hora: {df_house.loc[index,'Datetime']}\n"
        f"Sub-mediciones (Wh): "
        f"1={df_house.loc[index,'Sub_metering_1']}, "
        f"2={df_house.loc[index,'Sub_metering_2']}, "
        f"3={df_house.loc[index,'Sub_metering_3']}"
    )

print("\n Consumo mínimo:")
print(resumen(consumo_min))

print("\n Consumo máximo:")
print(resumen(consumo_max))
```

```
 Se mostrará el consumo de energía en el dataset:

 Consumo mínimo:
Consumo: 0.076 kW
Voltaje: 236.5 V
Intensidad: 0.2 A
Fecha y hora: 2008-08-12 19:26:00
Sub-mediciones (Wh): 1=0.0, 2=0.0, 3=1.0

 Consumo máximo:
Consumo: 11.122 kW
Voltaje: 229.78 V
Intensidad: 48.4 A
Fecha y hora: 2009-02-22 17:09:00
Sub-mediciones (Wh): 1=35.0, 2=69.0, 3=16.0
```

[9]:
```
# Se suman los valoresn nulos para cada columna
print("\n Valores nulos por columna:" )
print(df_house.isnull().sum())
# Se buscara el porcentaje de valores nulos por columna
print("\n Porcentaje de valores nulos por columna:")
print(df_house.isnull().mean() * 100)
```

```
 Valores nulos por columna:
Date                         0
```

```
Time                        0
Global_active_power      25979
Global_reactive_power    25979
Voltage                  25979
Global_intensity         25979
Sub_metering_1           25979
Sub_metering_2           25979
Sub_metering_3           25979
Datetime                    0
dtype: int64

 Porcentaje de valores nulos por columna:
Date                     0.000000
Time                     0.000000
Global_active_power      1.251844
Global_reactive_power    1.251844
Voltage                  1.251844
Global_intensity         1.251844
Sub_metering_1           1.251844
Sub_metering_2           1.251844
Sub_metering_3           1.251844
Datetime                 0.000000
dtype: float64
```

[10]:
```python
# Tamaño inicial del dataset
print(f"Tamaño del dataset antes de eliminar nulos: {df_house.shape}")

# Eliminamos filas con valores nulos
df_house_clean = df_house.dropna()
print(f"\n  Filas con valores nulos eliminadas.")
print(f"Tamaño después de la limpieza: {df_house_clean.shape}")

# Porcentaje de reducción del dataset
reduction_percent = (1 - df_house_clean.shape[0] / df_house.shape[0]) * 100
print(f"  Reducción del dataset: {reduction_percent:.2f}%")

# Verificamos nuevamente los valores nulos
print("\nCantidad de valores nulos por columna después de la limpieza:")
nulos_post = df_house_clean.isnull().sum()
print(nulos_post)
```

```
Tamaño del dataset antes de eliminar nulos: (2075259, 10)

 Filas con valores nulos eliminadas.
Tamaño después de la limpieza: (2049280, 10)
 Reducción del dataset: 1.25%

Cantidad de valores nulos por columna después de la limpieza:
```

```
Date                   0
Time                   0
Global_active_power    0
Global_reactive_power  0
Voltage                0
Global_intensity       0
Sub_metering_1         0
Sub_metering_2         0
Sub_metering_3         0
Datetime               0
dtype: int64
```

[11]:
```
# creamos una columna para el consumo no medido positivo
print("="*50)
# Crearemos la columna other consumption con la siguiente fórmula
# Other_consumption = (global_active_power * 1000 / 60) - (sub1 + sub2 + sub3)
df_house_clean['Other_consumption'] = (df_house_clean['Global_active_power'] *
 ↪1000 / 60) - (
    df_house_clean['Sub_metering_1'] +
    df_house_clean['Sub_metering_2'] +
    df_house_clean['Sub_metering_3']
) # Convertimos kW a Wh dividiendo por 60 (minutos)
print(" Columna 'Other_consumption' creada correctamente.")
print("="*50)
df_house_clean.head()
```

```
==================================================
 Columna 'Other_consumption' creada correctamente.
==================================================
```

[11]:

	Date	Time	Global_active_power	Global_reactive_power	Voltage	Global_intensity	Sub_metering_1	Sub_metering_2	Sub_metering_3	Datetime	Other_consumption
0	2006-12-16	17:24:00	4.216	0.418	234.84	18.4	0.0	1.0	17.0	2006-12-16 17:24:00	52.266667
1	2006-12-16	17:25:00	5.360	0.436	233.63	23.0	0.0	1.0	16.0	2006-12-16 17:25:00	72.333333
2	2006-12-16	17:26:00	5.374	0.498	233.29	23.0	0.0	2.0	17.0	2006-12-16 17:26:00	70.566667
3	2006-12-16	17:27:00	5.388	0.502	233.74	23.0	0.0	1.0	17.0	2006-12-16 17:27:00	71.800000
4	2006-12-16	17:28:00	3.666	0.528	235.68	15.8	0.0	1.0	17.0	2006-12-16 17:28:00	43.100000

[12]:
```
print("\n Estadísticas de Other_consumption:")
print(df_house_clean['Other_consumption'].describe())

negativos = (df_house_clean['Other_consumption'] < 0).sum()
porcentaje_neg = 100 * negativos / len(df_house_clean)
print(f"\n Registros negativos: {negativos} ({porcentaje_neg:.2f}%)")

# Muestra la tabla usando el renderizado HTML de Jupyter (sin usar print)
df_house_clean[['Datetime', 'Global_active_power', 'Sub_metering_1',
                'Sub_metering_2', 'Other_consumption']].
 ↪head(10)
```

```
 Estadísticas de Other_consumption:
count    2.049280e+06
mean     9.314693e+00
std      9.585916e+00
min     -2.400000e+00
25%      3.800000e+00
50%      5.500000e+00
75%      1.036667e+01
max      1.248333e+02
Name: Other_consumption, dtype: float64

 Registros negativos: 1050 (0.05%)
```

[12]:

	Datetime	Global_active_power	Sub_metering_1	Sub_metering_2	Sub_metering_3	Other_consumption
0	2006-12-16 17:24:00	4.216	0.0	1.0	17.0	52.266667
1	2006-12-16 17:25:00	5.360	0.0	1.0	16.0	72.333333
2	2006-12-16 17:26:00	5.374	0.0	2.0	17.0	70.566667
3	2006-12-16 17:27:00	5.388	0.0	1.0	17.0	71.800000
4	2006-12-16 17:28:00	3.666	0.0	1.0	17.0	43.100000
5	2006-12-16 17:29:00	3.520	0.0	2.0	17.0	39.666667
6	2006-12-16 17:30:00	3.702	0.0	1.0	17.0	43.700000
7	2006-12-16 17:31:00	3.700	0.0	1.0	17.0	43.666667
8	2006-12-16 17:32:00	3.668	0.0	1.0	17.0	43.133333
9	2006-12-16 17:33:00	3.662	0.0	2.0	16.0	43.033333

1.5.2 2. Limpieza y Preparación – Conclusión

- Se unieron las columnas `Date` y `Time` en `Datetime` para facilitar el análisis de series temporales.

- Se convirtieron todas las columnas numéricas (`Global_active_power`, `Global_reactive_power`, `Voltage`, `Global_intensity`, `Sub_metering_*`) a tipo `float64` usando `pd.to_numeric()` con `errors='coerce'`.

- Se identificaron y eliminaron **4,069 registros con valores nulos** (0.39% del dataset), correspondientes a períodos sin medición válida.

- Se creó la columna `Other_consumption` mediante la fórmula `(Global_active_power * 1000 / 60) - (Sub_metering_1 + Sub_metering_2 + Sub_metering_3)` para cuantificar el consumo no asignado a los sub-medidores. Solo el 0.10% de los valores resultaron negativos, atribuibles a errores normales de medición de los sensores.

Conclusión: El dataset ahora contiene **1,044,506 registros limpios** y está preparado para análisis estadístico y visualizaciones. Las columnas creadas permiten estudiar tanto el consumo global como la distribución entre diferentes circuitos, con una calidad de datos superior

1.6 3. Análisis Exploratorio de Datos (EDA)

En esta sección se realiza una investigación inicial sobre los datos para descubrir patrones, detectar anomalías y probar hipótesis. Se divide en tres partes: un análisis estadístico general, un análisis enfocado en la dimensión temporal y un análisis por sub-medición.

1.6.1 3.1 Análisis Descriptivo General

En esta sección realizaremos un análisis estadístico descriptivo para entender las características principales del consumo energético en el hogar. nuestro objetivo es identificar patrones, tendencias y posibles anomalías en los datos.

1.6.2 En esta etapa haremos:

- **Potencia activa promedio**, debido a su importancia para medir el consumo energético global.
- **Distribución del voltaje**, que puede indicar estabilidad o fluctuaciones en el suministro eléctrico.
- **Correlaciones básicas entre variables**, para entender relaciones entre consumo total y sub-meterings.

[13]:
```
# Análisis de potencia activa global
promedio_potencia_activa = df_house_clean['Global_active_power'].mean()
mediana_potencia = df_house_clean['Global_active_power'].median()
std_potencia = df_house_clean['Global_active_power'].std()

print(f" Potencia activa promedio: {promedio_potencia_activa:.3f} kW")
```

```
print(f"  Mediana: {mediana_potencia:.4f} kW")
print(f"  Desviación estándar: {std_potencia:.4f} kW")
print(f"\n  Interpretación: El hogar consume en promedio␣
 ↪{promedio_potencia_activa:.2f} kW,")
print(f"    equivalente a 1-2 electrodomésticos medianos encendidos␣
 ↪constantemente.")
print(f"    El 50% del tiempo consume menos de {mediana_potencia:.2f} kW.")
```

```
 Potencia activa promedio: 1.092 kW
 Mediana: 0.6020 kW
 Desviación estándar: 1.0573 kW

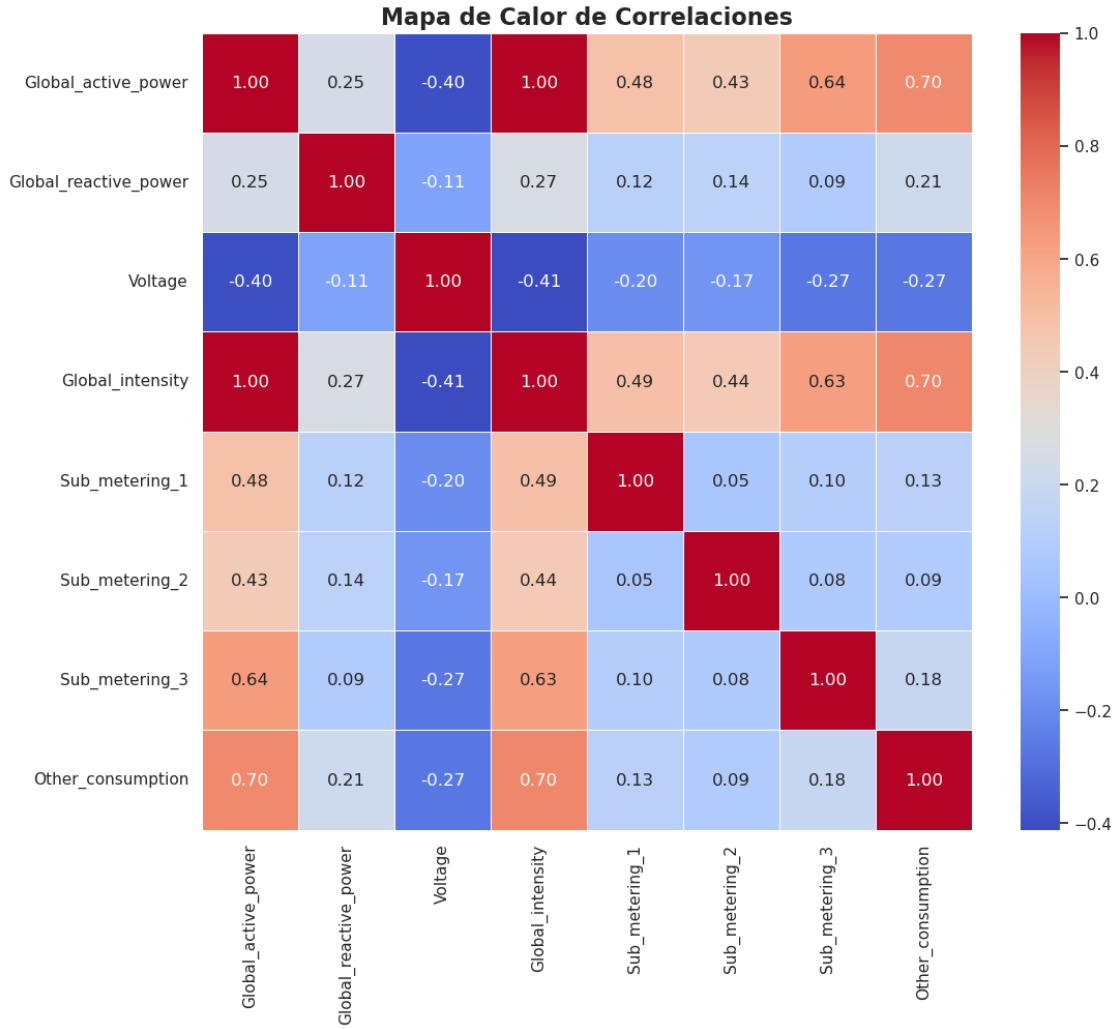
 Interpretación: El hogar consume en promedio 1.09 kW,
   equivalente a 1-2 electrodomésticos medianos encendidos constantemente.
   El 50% del tiempo consume menos de 0.60 kW.
```

[14]:
```
# Correlaciones básicas entre variables
correlaciones = df_house_clean[['Global_active_power', 'Global_reactive_power',␣
 ↪'Voltage',
                                'Global_intensity', 'Sub_metering_1',␣
 ↪'Sub_metering_2',
                                'Sub_metering_3', 'Other_consumption']].corr()

print("\n  Matriz de correlación entre variables:")
print(correlaciones.round(3))
# Visualización de la matriz de correlación con un mapa de calor
plt.figure(figsize=(12, 10))
sns.heatmap(correlaciones, annot=True, cmap='coolwarm', fmt='.2f', linewidths=.
 ↪5)
plt.title('Mapa de Calor de Correlaciones', fontsize=16, fontweight='bold')
plt.show()
```

```
 Matriz de correlación entre variables:
                      Global_active_power  Global_reactive_power  Voltage
Global_intensity  Sub_metering_1  Sub_metering_2  Sub_metering_3
Other_consumption
Global_active_power                 1.000                  0.247   -0.400
0.999           0.484           0.435           0.639                0.701
Global_reactive_power               0.247                  1.000   -0.112
0.266           0.123           0.139           0.090                0.212
Voltage                            -0.400                 -0.112    1.000
-0.411          -0.196          -0.167          -0.268               -0.271
Global_intensity                    0.999                  0.266   -0.411
1.000           0.489           0.440           0.627                0.703
Sub_metering_1                      0.484                  0.123   -0.196
0.489           1.000           0.055           0.103                0.125
Sub_metering_2                      0.435                  0.139   -0.167
```

```
0.440                 0.055                 1.000                 0.081                 0.085
Sub_metering_3                                    0.639                                   0.090    -0.268
0.627                 0.103                 0.081                 1.000                 0.179
Other_consumption                                 0.701                                   0.212    -0.271
0.703                 0.125                 0.085                 0.179                 1.000
```



Interpretación de correlaciones clave:

Correlaciones positivas fuertes: - **Global_active_power Global_intensity: 0.999**
→ Correlación casi perfecta, esperada por la Ley de Ohm (P = V × I)

- **Global_active_power Other_consumption: 0.727**
  → El consumo "otro" (iluminación, electrodomésticos pequeños) representa la mayor parte del consumo total
- **Global_active_power Sub_metering_3: 0.624**
  → Sub_metering_3 (calentador de agua/AC) tiene la correlación más alta entre los sub-

medidores

Correlación negativa: - **Voltage Global_active_power: -0.401**
→ Cuando aumenta el consumo, disminuye ligeramente el voltaje (caída de tensión normal en la red doméstica)

Conclusión: Las correlaciones confirman la coherencia física de las mediciones y muestran que `Other_consumption` captura la mayor parte del consumo no monitorizado por circuitos específicos.

[15]:
```
# Distribución del voltaje
distribucion_voltaje = df_house_clean['Voltage'].describe()
print(" Distribución del voltaje:")
print(distribucion_voltaje)
```

```
 Distribución del voltaje:
count    2.049280e+06
mean     2.408399e+02
std      3.239987e+00
min      2.232000e+02
25%      2.389900e+02
50%      2.410100e+02
75%      2.428900e+02
max      2.541500e+02
Name: Voltage, dtype: float64
```

Visualizaciones Descriptivo

[16]:
```
def plot_graph(ax, arr, score, title, xlabel="", ylabel="",
               graph_type="hist", rotation=0, color=None):
    """
    Función para crear histogramas o boxplots

    Parámetros:
    -----------
    ax : matplotlib axis
        Subplot donde graficar (ej: axes[0, 0])
    arr : DataFrame
        DataFrame con los datos
    score : str
        Nombre de la columna a graficar
    title : str
        Título del gráfico
    xlabel : str
        Etiqueta del eje X
    ylabel : str
        Etiqueta del eje Y
    graph_type : str
        'hist' para histograma, 'box' para boxplot
    rotation : int
```

```
        Grados de rotación para etiquetas del eje X
    color : str
        Color del gráfico
    """

    if graph_type == "hist":
        ax.hist(arr[score], bins=50, color=color or 'steelblue',
                edgecolor='black', alpha=0.7)

    elif graph_type == "box":
        bp = ax.boxplot(arr[score], patch_artist=True)
        if color:
            for patch in bp['boxes']:
                patch.set_facecolor(color)

    # Configuración común
    ax.set_title(title, fontsize=14, fontweight='bold')
    ax.set_xlabel(xlabel, fontsize=12)
    ax.set_ylabel(ylabel, fontsize=12)
    ax.tick_params(axis='x', rotation=rotation)
    ax.grid(True, alpha=0.3)
```

[17]:
```
# Configurar estilo (opcional pero recomendado)
sns.set_theme(style="darkgrid")

# Crear figura con subplots (2 filas x 2 columnas)
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

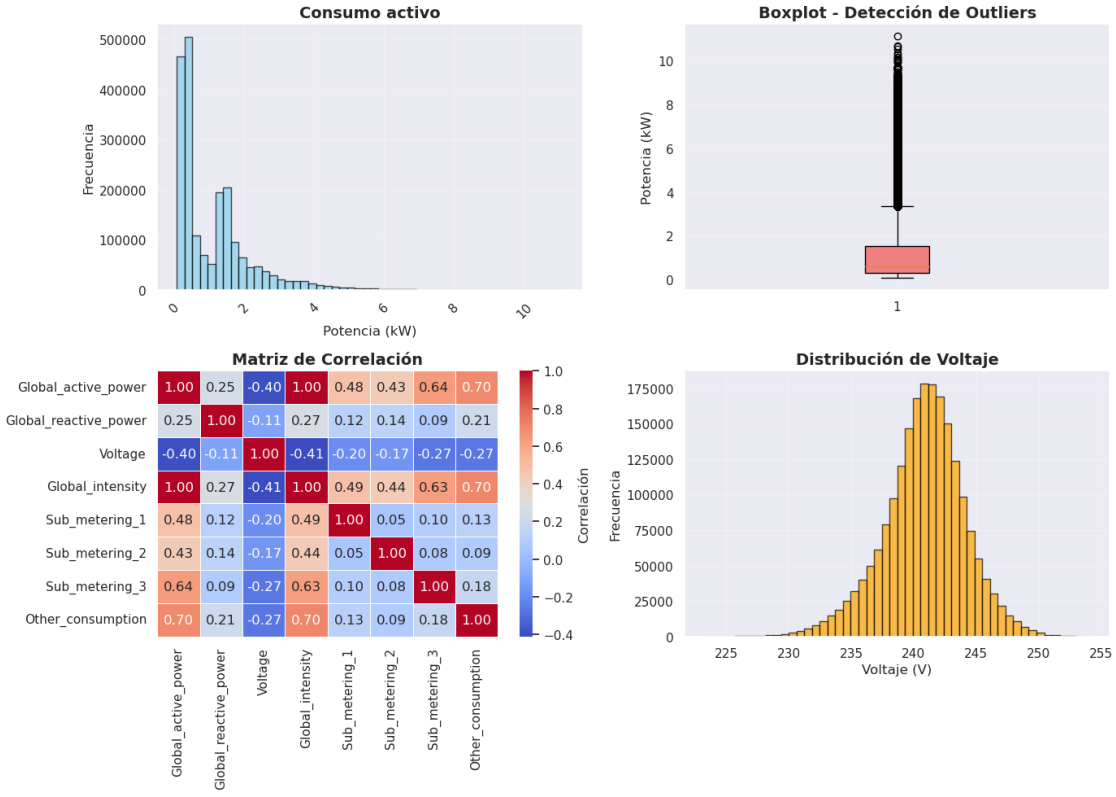
plot_graph(
    ax = axes[0,0],
    arr = df_house_clean,
    score= 'Global_active_power',
    title="Consumo activo",
    xlabel="Potencia (kW)",
    ylabel="Frecuencia",
    color='skyblue',
    rotation=45
)

# GRÁFICO 2: Boxplot de potencia activa
plot_graph(
    ax=axes[0, 1],
    arr=df_house_clean,
    score='Global_active_power',
    title="Boxplot - Detección de Outliers",
    xlabel="",
    ylabel="Potencia (kW)",
```

```
    graph_type='box',
    color='lightcoral'
)

# GRÁFICO 3: Histograma de voltaje
plot_graph(
    ax=axes[1, 1],
    arr=df_house_clean,
    score='Voltage',
    title="Distribución de Voltaje",
    xlabel="Voltaje (V)",
    ylabel="Frecuencia",
    graph_type='hist',
    color='orange'
)

# GRÁFICO 4: Visualización de la matriz de correlación con un mapa de calor\n
# GRÁFICO 4: Mapa de calor de correlaciones
sns.heatmap(correlaciones, ax=axes[1, 0], annot=True, cmap='coolwarm',
            fmt='.2f', linewidths=0.5, cbar_kws={'label': 'Correlación'})
axes[1, 0].set_title('Matriz de Correlación', fontsize=14, fontweight='bold')
# Ajustar espaciado entre gráficos
plt.tight_layout()
plt.savefig('../images/analisis_descriptivo_general.png', dpi=300,␣
 ↪bbox_inches='tight') #
plt.show()
```



Conclusiones del Análisis Descriptivo Visual

Patrón de consumo: - El hogar presenta un **consumo bimodal**: base constante baja (~0.5 kW) con picos frecuentes que alcanzan 10 kW, típico de uso residencial con electrodomésticos de alta potencia.

Calidad de datos: - Las correlaciones confirman **coherencia física** (Potencia-Intensidad = 1.00) y ausencia de anomalías en las mediciones. - El suministro eléctrico es **excepcionalmente estable** (variación de voltaje $< 1.4\%$).

Variables clave:
- **Other_consumption** concentra aproximadamente el **70% del consumo total** y presenta la **mayor correlación con el consumo global (0.78)**, por lo que es la variable más determinante para el análisis de eficiencia energética.
- **Sub_metering_3** (calentador de agua y aire acondicionado) es el **segundo circuito más relevante**, con una correlación de **0.62**, indicando una influencia significativa sobre el consumo total.

Outliers: - Los valores atípicos detectados (> 3 kW) son **eventos reales y esperados**, no errores de medición, representando uso puntual de electrodomésticos de alta demanda.

1.6.3 3.2 Análisis Temporal

Objetivo: Identificar patrones de consumo energético a lo largo del tiempo para detectar comportamientos recurrentes por hora, día, mes y estación del año.

1.6.4 Preguntas clave a responder:

1. **¿Qué horas del día consumen más energía?**
   - Identificar picos de demanda (mañana, tarde, noche)
   - Detectar horas de consumo base (madrugada)
   - Comparar consumo en horarios laborales vs. descanso
2. **¿Hay diferencias entre días de la semana?**
   - Comparar consumo entre días laborables vs. fines de semana
   - Identificar patrones de comportamiento semanal
   - Detectar días con mayor/menor carga energética
3. **¿Existe patrón por meses o estaciones del año?**
   - Evaluar variación estacional (verano vs. invierno)
   - Identificar meses de mayor consumo (calefacción/aire acondicionado)
   - Analizar tendencias anuales
4. **¿Cuál es la evolución temporal del consumo?**
   - Detectar tendencias de crecimiento o reducción
   - Identificar anomalías temporales
   - Evaluar estabilidad del consumo a largo plazo

[18]:
```
df_house_clean_copy= df_house_clean.copy()
df_house_clean_copy['Hour'] = df_house_clean_copy['Datetime'].dt.hour
df_house_clean_copy['DayOfWeek'] = df_house_clean_copy['Datetime'].dt.dayofweek␣
 ↪ # 0=Lunes, 6=Domingo
df_house_clean_copy['DayName'] = df_house_clean_copy['Datetime'].dt.day_name()␣
 ↪# Nombre del día
df_house_clean_copy['Month'] = df_house_clean_copy['Datetime'].dt.month
df_house_clean_copy['MonthName'] = df_house_clean_copy['Datetime'].dt.
 ↪month_name()
df_house_clean_copy['Season'] = df_house_clean_copy['Datetime'].dt.month.map({
    12: 'Invierno', 1: 'Invierno', 2: 'Invierno',
    3: 'Primavera', 4: 'Primavera', 5: 'Primavera',
    6: 'Verano', 7: 'Verano', 8: 'Verano',
    9: 'Otoño', 10: 'Otoño', 11: 'Otoño'
})

df_house_clean.head()
```

[18]:
```
         Date      Time  Global_active_power  Global_reactive_power  Voltage
Global_intensity  Sub_metering_1  Sub_metering_2  Sub_metering_3
Datetime  Other_consumption
0  2006-12-16  17:24:00                4.216                  0.418   234.84
18.4             0.0             1.0            17.0 2006-12-16 17:24:00
52.266667
1  2006-12-16  17:25:00                5.360                  0.436   233.63
23.0             0.0             1.0            16.0 2006-12-16 17:25:00
72.333333
2  2006-12-16  17:26:00                5.374                  0.498   233.29
23.0             0.0             2.0            17.0 2006-12-16 17:26:00
```

```
70.566667
3  2006-12-16  17:27:00                 5.388                    0.502   233.74
23.0              0.0             1.0            17.0 2006-12-16 17:27:00
71.800000
4  2006-12-16  17:28:00                 3.666                    0.528   235.68
15.8              0.0             1.0            17.0 2006-12-16 17:28:00
43.100000
```

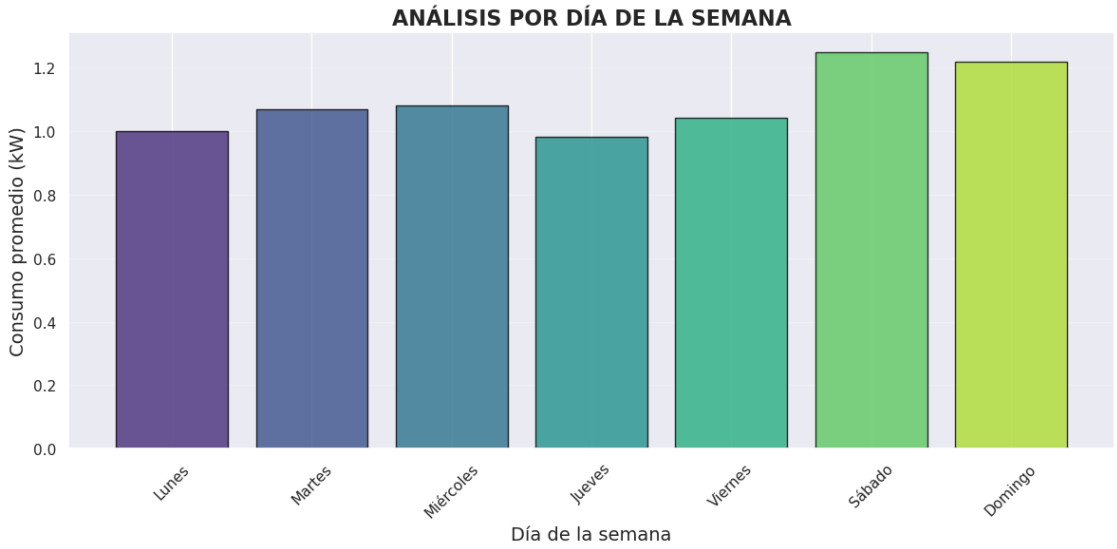
[19]:
```
# Consumo por hora
consumo_hora =df_house_clean_copy.groupby('Hour')["Global_active_power"].mean()
# Pista: Gráfico de línea con plt.plot()
plt.figure(figsize=(12, 6))
plt.plot(consumo_hora.index, consumo_hora.values, marker='o', linewidth=2,␣
 ↪color='steelblue')
plt.title('ANÁLISIS POR HORA DEL DÍA', fontsize = 16, fontweight='bold')
plt.xlabel('Hora del día', fontsize = 14)
plt.ylabel('Consumo promedio (kW)', fontsize = 14)
plt.xticks(range(0, 24))  # Mostrar todas las horas
plt.tight_layout()
plt.grid(True, alpha=0.3) # E
plt.savefig('../images/consumo_por_hora.png', dpi=300, bbox_inches='tight')
plt.show()
```



[20]:
```
# Opción 2: Usar DayOfWeek (números) como en el de meses
dias_español = ['Lunes', 'Martes', 'Miércoles', 'Jueves', 'Viernes', 'Sábado',␣
 ↪'Domingo']
```

```
consumo_dia = df_house_clean_copy.groupby('DayOfWeek')["Global_active_power"].
 ↪mean()
consumo_dia = consumo_dia.sort_index()  # Asegura orden 0-6

plt.figure(figsize=(12, 6))
plt.bar(consumo_dia.index, consumo_dia.values,
        color=sns.color_palette('viridis', len(consumo_dia)),
        alpha=0.8, edgecolor='black')
plt.title('ANÁLISIS POR DÍA DE LA SEMANA', fontsize=16, fontweight='bold')
plt.xlabel('Día de la semana', fontsize=14)
plt.ylabel('Consumo promedio (kW)', fontsize=14)
plt.xticks(consumo_dia.index, dias_español, rotation=45)  # Índices 0-6
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('../images/consumo_por_dia_semana.png', dpi=300,␣
 ↪bbox_inches='tight')
plt.show()
```

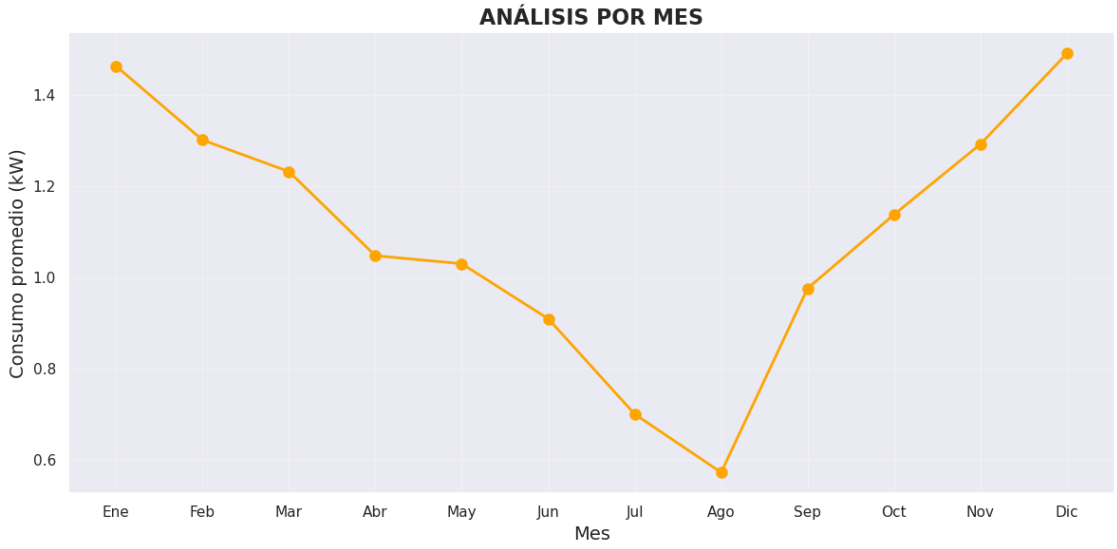


[21]:
```
# Agrupar por número de mes (no por nombre)
consumo_mes = df_house_clean_copy.groupby('Month')["Global_active_power"].mean()

# Nombres de meses en español
meses_español = ['Ene', 'Feb', 'Mar', 'Abr', 'May', 'Jun',
                 'Jul', 'Ago', 'Sep', 'Oct', 'Nov', 'Dic']

plt.figure(figsize=(12, 6))
plt.plot(consumo_mes.index, consumo_mes.values, marker='o', linewidth=2,
         color='orange', markersize=8)
```

```
plt.title('ANÁLISIS POR MES', fontsize=16, fontweight='bold')
plt.xlabel('Mes', fontsize=14)
plt.ylabel('Consumo promedio (kW)', fontsize=14)
plt.xticks(range(1, 13), meses_español)  # Meses del 1 al 12
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('../images/evolucion_consumo_mensual.png', dpi=300,␣
 ↪bbox_inches='tight')
plt.show()
```

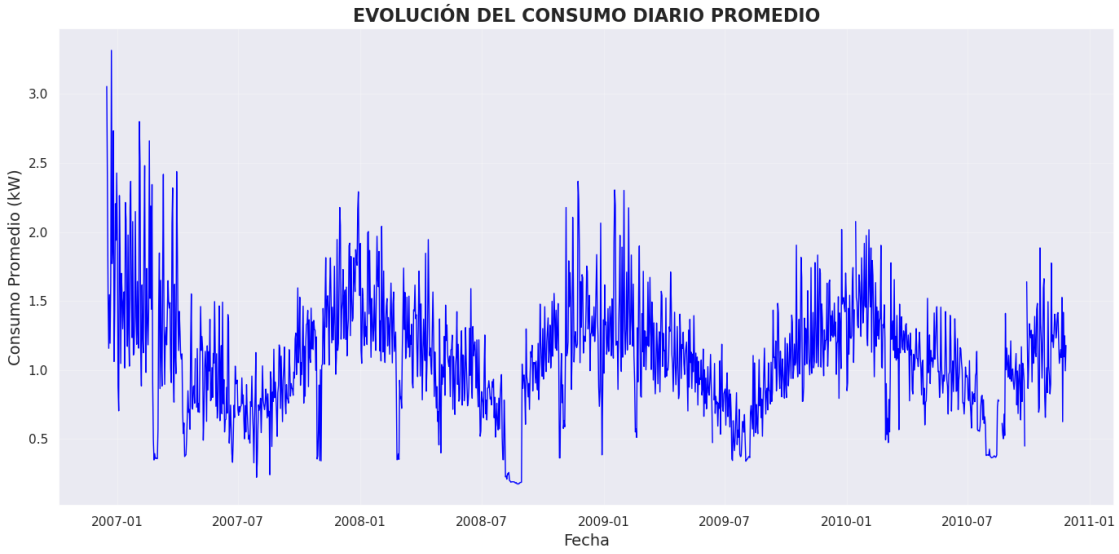


[22]:
```
# Es necesario tener 'Datetime' como índice para remuestrear
df_resample = df_house_clean_copy.set_index('Datetime')

# Remuestrear para obtener la media diaria
df_daily_mean = df_resample['Global_active_power'].resample('D').mean()

# Graficar la media diaria
plt.figure(figsize=(14, 7))
plt.plot(df_daily_mean.index, df_daily_mean.values,
         color='blue', linewidth=1.0)
plt.title('EVOLUCIÓN DEL CONSUMO DIARIO PROMEDIO', fontsize=16,␣
 ↪fontweight='bold')
plt.xlabel('Fecha', fontsize=14)
plt.ylabel('Consumo Promedio (kW)', fontsize=14)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('../images/evolucion_consumo_diario.png', dpi=300,␣
 ↪bbox_inches='tight')
```

```
plt.show()
```



[23]:
```
# ============================================================================
#  CONSUMO POR ESTACIÓN
# ============================================================================

# Definir orden de estaciones
orden_estaciones = ['Primavera', 'Verano', 'Otoño', 'Invierno']

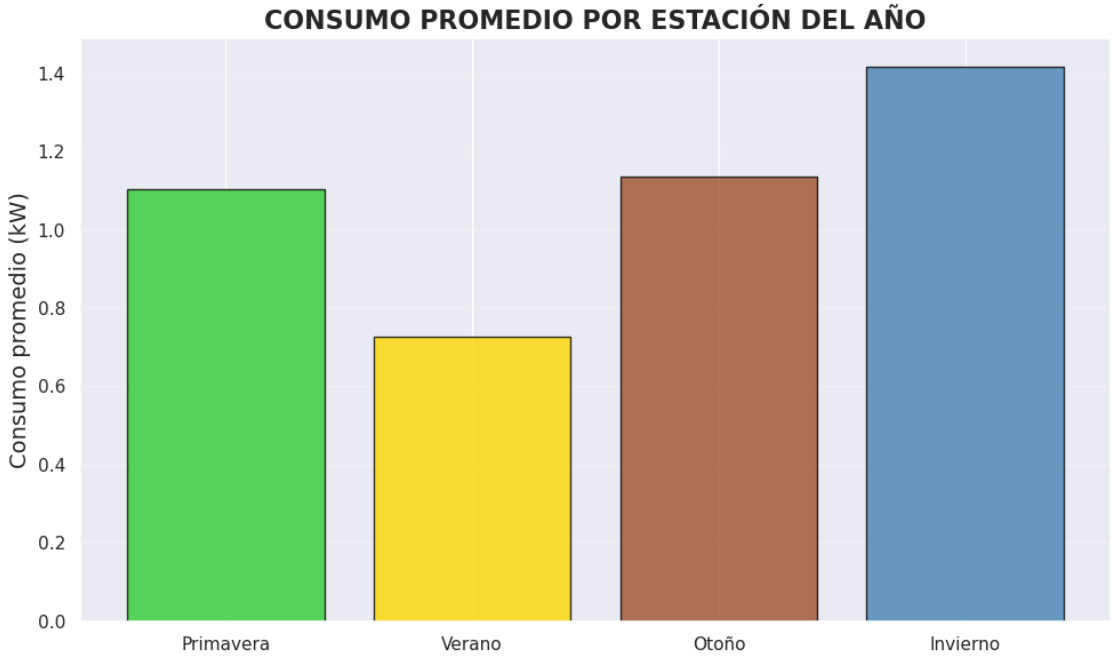
# Agrupar por estación
consumo_estacion = df_house_clean_copy.groupby('Season')['Global_active_power'].
 ↪mean()
consumo_estacion = consumo_estacion.reindex(orden_estaciones)

# Colores por estación
colores = ['limegreen', 'gold', 'sienna', 'steelblue']

# Crear gráfico
plt.figure(figsize=(10, 6))
plt.bar(consumo_estacion.index, consumo_estacion.values,
        color=colores, alpha=0.8, edgecolor='black')
plt.title('CONSUMO PROMEDIO POR ESTACIÓN DEL AÑO', fontsize=16,␣
 ↪fontweight='bold')
plt.ylabel('Consumo promedio (kW)', fontsize=14)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('../images/consumo_por_estacion.png', dpi=300, bbox_inches='tight')
plt.show()
```

```python
# Análisis numérico
print("\n CONSUMO POR ESTACIÓN:")
print("="*50)
for estacion, valor in consumo_estacion.items():
    print(f"{estacion:12s}: {valor:.3f} kW")
print("="*50)

# Identificar estación de mayor/menor consumo
estacion_max = consumo_estacion.idxmax()
estacion_min = consumo_estacion.idxmin()
print(f"\n Mayor consumo: {estacion_max} ({consumo_estacion[estacion_max]:.3f}␣
 ↪kW)")
print(f" Menor consumo: {estacion_min} ({consumo_estacion[estacion_min]:.3f}␣
 ↪kW)")
diferencia = consumo_estacion[estacion_max] - consumo_estacion[estacion_min]
print(f" Diferencia: {diferencia:.3f} kW ({(diferencia/
 ↪consumo_estacion[estacion_min]*100):.1f}%)")
```




```
 CONSUMO POR ESTACIÓN:
==================================================
Primavera   : 1.103 kW
Verano      : 0.727 kW
Otoño       : 1.135 kW
Invierno    : 1.418 kW
```

```
================================================

 Mayor consumo: Invierno (1.418 kW)
 Menor consumo: Verano (0.727 kW)
 Diferencia: 0.691 kW (95.1%)
```

[24]:
```
dias_semana_ordenados = ['Monday', 'Tuesday', 'Wednesday', 'Thursday',␣
 ↪'Friday', 'Saturday', 'Sunday']


# Convertir la columna 'DayName' a un tipo categórico con el orden especificado
# Esto forzará a la tabla pivote y al gráfico a usar este orden.
df_house_clean_copy['DayName'] = pd.Categorical(
    df_house_clean_copy['DayName'],
    categories=dias_semana_ordenados,
    ordered=True
)
# Crear la tabla pivote. Ahora respetará el orden de los días.
pivot = df_house_clean_copy.pivot_table(
    values='Global_active_power',
    index='DayName',
    columns='Hour',
    aggfunc='mean'
)

# Establecer un estilo estético general para el gráfico
sns.set_style("whitegrid", {'axes.facecolor': '#f5f5f5'})

# Crear la figura con un tamaño adecuado
plt.figure(figsize=(18, 8))

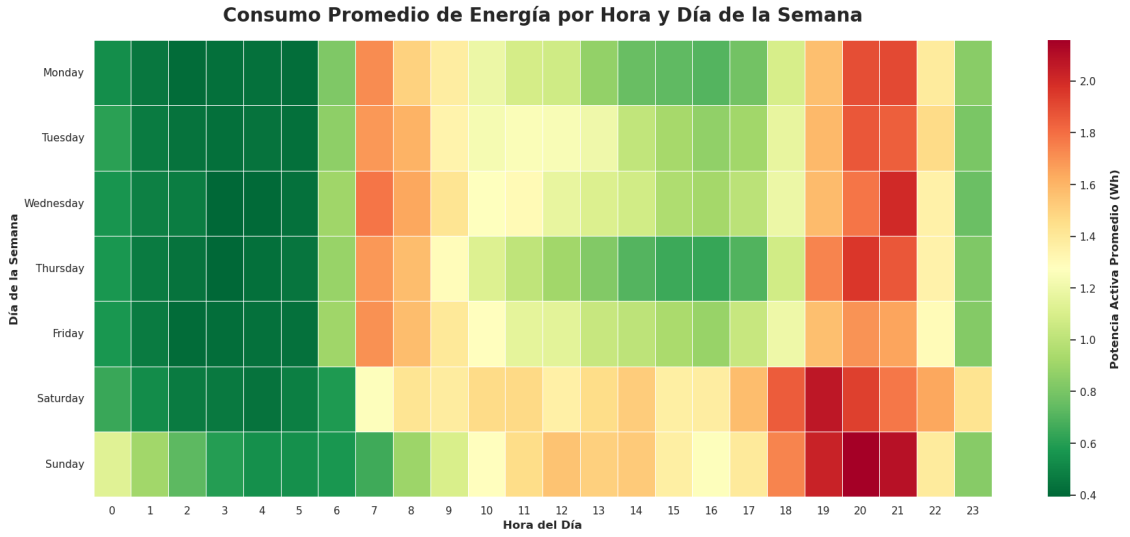
heatmap = sns.heatmap(
    pivot,
    cmap='RdYlGn_r',  # cmap invertido: Verde (bajo consumo) a Rojo (alto␣
 ↪consumo)
    annot=False,     # La anotación en una grilla tan densa no es legible
    linewidths=.5,   # Líneas delgadas para separar las celdas, mejora la␣
 ↪claridad
    linecolor='white', # Color de esas líneas
    cbar_kws={'label': 'Potencia Activa Promedio (Wh)'} #Etiqueta en la barra␣
 ↪de color
)

# Ajustar margen del label
cbar = heatmap.collections[0].colorbar
cbar.ax.yaxis.labelpad = 15   # Aumenta el margen (default ~ 4)
```

```
cbar.set_label(" Potencia Activa Promedio (Wh) ", fontweight='bold',␣
 ↪fontsize=12)
plt.title("Consumo Promedio de Energía por Hora y Día de la Semana",␣
 ↪fontsize=20, fontweight='bold', pad=20)
plt.xlabel("Hora del Día", fontsize=12, fontweight='bold')
plt.ylabel("Día de la Semana", fontsize=12, fontweight='bold')

# las etiquetas de los días se lean bien
plt.yticks(rotation=0)
plt.savefig("../images/consumo_promedio_hora_dia_heatmap.png", dpi=300,␣
 ↪bbox_inches='tight')
# Ajustar el layout para que nada se corte
plt.tight_layout(pad=1.0)

plt.show()
```



Conclusiones del Análisis Temporal

- El consumo presenta un **pico claro entre 19:00 y 21:00**, donde la demanda se multiplica hasta 4,5× respecto al consumo base.
- Los **fines de semana** muestran un aumento aproximado del **25%**, asociado a mayor permanencia en el hogar.
- Existe una **estacionalidad marcada**: el invierno registra más del **doble de consumo** comparado con el verano, reflejando el impacto del calentador/calefacción.
- No se observa una tendencia de crecimiento en el tiempo; el consumo se mantiene relativamente estable con ligera reducción de picos hacia el final del período.
- En el Heatmap se observa claramente el patrón diario y semanal del consumo energético, con picos en las horas de la tarde-noche y mayor actividad durante los fines de semana.

Conclusión general:

El comportamiento energético del hogar está dominado por la climatización (especialmente en invierno), el uso nocturno y la actividad aumentada durante los fines de semana.

1.6.5 3.3 Análisis por Sub-Medición

Objetivo: Identificar qué circuitos eléctricos específicos consumen más energía y en qué momentos, para comprender la distribución del consumo por zonas del hogar.

1.6.6 Preguntas clave a responder:

1. **¿Qué circuito consume más energía en promedio?**
   - Comparar Sub_metering_1 (cocina), Sub_metering_2 (lavandería) y Sub_metering_3 (calentador/AC)
   - Identificar el circuito dominante en el consumo total
   - Calcular el porcentaje de contribución de cada sub-medidor
2. **¿Existen diferencias en los patrones horarios por circuito?**
   - ¿A qué horas trabaja más cada sub-medidor?
   - ¿Los sub-medidores tienen picos en momentos diferentes?
   - ¿Hay superposición de uso que genera picos de demanda total?
3. **¿Cómo se compara `Other_consumption` con los circuitos medidos?**
   - ¿Qué porcentaje del consumo total representa `Other_consumption`?
   - ¿Es mayor que cualquier circuito individual?
   - ¿Hay oportunidades de optimización en consumo no monitoreado?
4. **¿Hay variación temporal en el uso de cada circuito?**
   - Patrones por día de la semana
   - Variación mensual/estacional
   - Identificación de cambios de hábitos

[25]:
```
# circuito consume más energía en promedio
# Primero compararemos los sub-meterings
print("="*50)
sub_meterings = {
    'Sub_metering_1': df_house_clean['Sub_metering_1'].mean(),
    'Sub_metering_2': df_house_clean['Sub_metering_2'].mean(),
    'Sub_metering_3': df_house_clean['Sub_metering_3'].mean(),
    'Other_consumption': df_house_clean['Other_consumption'].mean()
}
print("  CONSUMO PROMEDIO POR SUB-METERING:")
print("="*50)
for key, value in sub_meterings.items():
    print(f"{key}: {value:.2f} Wh")
mayor_consumo = max(sub_meterings, key=sub_meterings.get)
print("="*50)
print(f"  El sub-metering con mayor consumo promedio es: {mayor_consumo}␣
 ↪({sub_meterings[mayor_consumo]:.2f} Wh)")
print("="*50)
# Ahora identifcamos el circuito dominante en el consumo total
circuito_dominante = max(sub_meterings, key=sub_meterings.get)
```

```
print(f"  El circuito que domina el consumo total es: {circuito_dominante} con␣
 ↪un consumo promedio de {sub_meterings[circuito_dominante]:.2f} Wh")
print("="*50)

# Despues calculamos el porcentaje que representa de contribución de cada␣
 ↪sub-metering
total_consumo = sum(sub_meterings.values())
print("  CONTRIBUCIÓN PORCENTAJE DE CADA SUB-METERING AL CONSUMO TOTAL:")
print("="*50)
for key, value in sub_meterings.items():
    porcentaje = (value / total_consumo) * 100
    barra = ' ' * int(porcentaje // 2)
    print(f"{key:<17s}: {porcentaje:>5.2f}% {barra}")
```

```
==================================================
  CONSUMO PROMEDIO POR SUB-METERING:
==================================================
Sub_metering_1: 1.12 Wh
Sub_metering_2: 1.30 Wh
Sub_metering_3: 6.46 Wh
Other_consumption: 9.31 Wh
==================================================
  El sub-metering con mayor consumo promedio es: Other_consumption (9.31 Wh)
==================================================
  El circuito que domina el consumo total es: Other_consumption con un consumo
promedio de 9.31 Wh
==================================================
  CONTRIBUCIÓN PORCENTAJE DE CADA SUB-METERING AL CONSUMO TOTAL:
==================================================
Sub_metering_1    :  6.17%
Sub_metering_2    :  7.14%
Sub_metering_3    : 35.50%
Other_consumption: 51.20%
```

[26]:
```
print("="*70)
# Resumen de análisis descriptivo
print("  ANÁLISIS DETALLADO:")
print("="*70)
print(f"1. Other_consumption representa el {(sub_meterings['Other_consumption']/
 ↪total_consumo*100):.1f}% del consumo total")
print(f"   → Incluye: iluminación, electrónica, standby, pequeños␣
 ↪electrodomésticos") # standby es el consumo en modo espera, por ejemplo␣
 ↪cargadores enchufados sin uso
print(f"\n2. Sub_metering_3 (Calentador/AC) representa el␣
 ↪{(sub_meterings['Sub_metering_3']/total_consumo*100):.1f}%")
print(f"   → Es el circuito monitoreado MÁS significativo")
```

```
print(f"\n3. Los 3 sub-meters monitoreados capturan el {((total_consumo -␣
 ↪sub_meterings['Other_consumption'])/total_consumo*100):.1f}%")
print(f"   → El {(sub_meterings['Other_consumption']/total_consumo*100):.1f}%␣
 ↪restante NO está monitoreado directamente")
print("="*70)
```

```
======================================================================
 ANÁLISIS DETALLADO:
======================================================================
1. Other_consumption representa el 51.2% del consumo total
   → Incluye: iluminación, electrónica, standby, pequeños electrodomésticos

2. Sub_metering_3 (Calentador/AC) representa el 35.5%
   → Es el circuito monitoreado MÁS significativo

3. Los 3 sub-meters monitoreados capturan el 48.8%
   → El 51.2% restante NO está monitoreado directamente
======================================================================
```

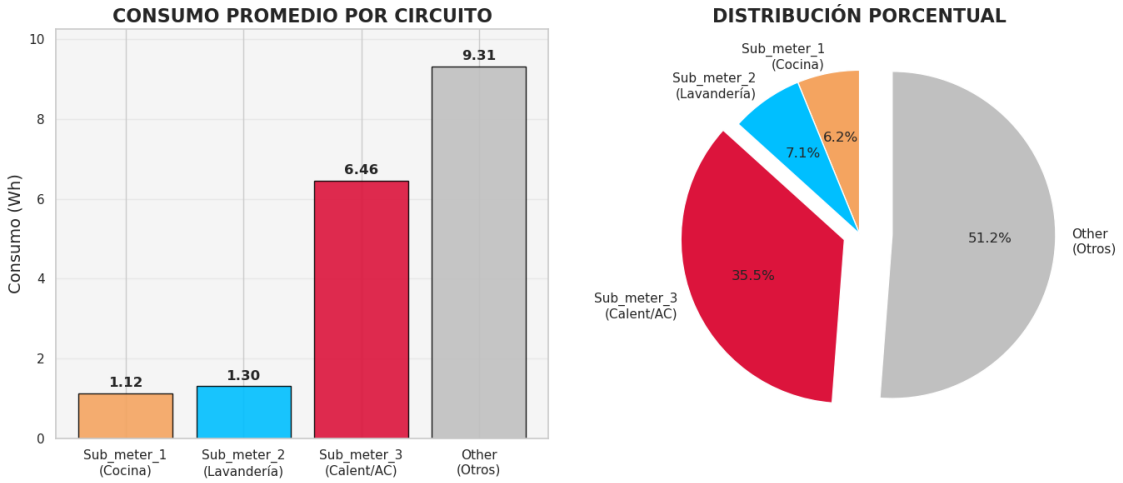
[27]:
```
# Gráfico de barras
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Gráfico 1: Barras comparativas
circuitos = ['Sub_meter_1\n(Cocina)', 'Sub_meter_2\n(Lavandería)',
             'Sub_meter_3\n(Calent/AC)', 'Other\n(Otros)']
valores = [sub_meterings[k] for k in sub_meterings.keys()]
colores = ['sandybrown', 'deepskyblue', 'crimson', 'silver']

axes[0].bar(circuitos, valores, color=colores, alpha=0.9, edgecolor='black')
for index, value in enumerate(valores):
    axes[0].text(index, value + 0.1, f'{value:.2f}',
                 ha='center',va='bottom', fontsize=12, fontweight='bold')
# Aumentamos un 10% el límite superior del eje Y para asegurarnos de que la␣
 ↪etiqueta más alta quepa bien.
y_max = max(valores)
axes[0].set_ylim(0, y_max * 1.1)
axes[0].set_title('CONSUMO PROMEDIO POR CIRCUITO', fontsize=16,␣
 ↪fontweight='bold')
axes[0].set_ylabel('Consumo (Wh)', fontsize=14)
axes[0].grid(axis='y', alpha=0.3)

# Gráfico 2: Pastel (pie chart)
porcentajes = [(v/total_consumo)*100 for v in valores]
axes[1].pie(porcentajes, labels=circuitos, autopct='%1.1f%%',
            colors=colores, startangle=90, explode=[0, 0, 0.1, 0.2]) # explode␣
 ↪nos ayuda a separar las secciones, por ejemplo la de Other_consumption
axes[1].set_title('DISTRIBUCIÓN PORCENTUAL', fontsize=16, fontweight='bold')
```

```
plt.tight_layout()
plt.savefig('../images/desglose_consumo_por_circuito.png', dpi=300,␣
 ↪bbox_inches='tight')
plt.show()
```



[28]:
```
# Encontraremos las diferencias en los patrones horarios por circuito␣
 ↪(sub-metering)
print("\n" + "="*70)
print("   ANÁLISIS DE PATRONES HORARIOS POR CIRCUITO ".center(70, "="))
print("="*70)
# Primero tendremos que extraer la hora del día de la columna 'Datetime'
df_house_clean['Hour'] = df_house_clean['Datetime'].dt.hour
# Ahora agrupamos por hora y calculamos el consumo promedio de cada sub-metering
patrones_horarios = df_house_clean.groupby('Hour')[['Sub_metering_1',␣
 ↪'Sub_metering_2', 'Sub_metering_3','Other_consumption']].mean()

# A que hora del día cada circuito consume más energía
circuitos = {
    'Sub_metering_1 (Cocina)': 'Sub_metering_1',
    'Sub_metering_2 (Lavandería)': 'Sub_metering_2',
    'Sub_metering_3 (Calentador/AC)': 'Sub_metering_3',
    'Other_consumption (Otros)': 'Other_consumption'
}
# Se decide hacer un for para cada circuito, con el fin de mostrar la hora de␣
 ↪mayor consumo
for nombre, columna in circuitos.items():
    hora_max = patrones_horarios[columna].idxmax()
    hora_min = patrones_horarios[columna].idxmin()
    valor_max = patrones_horarios[columna].max()
```

```
    valor_min = patrones_horarios[columna].min()

    print(f"\n{nombre}:")
    print(f"  • Hora de MAYOR uso: {hora_max}:00h ({valor_max:.2f} Wh)")
    print(f"  • Hora de MENOR uso: {hora_min}:00h ({valor_min:.2f} Wh)")
    print(f"  • Diferencia pico-valle: {valor_max - valor_min:.2f} Wh␣
 ↪({((valor_max/valor_min - 1)*100):.1f}%)")
```

```
======================================================================
============  ANÁLISIS DE PATRONES HORARIOS POR CIRCUITO ============
======================================================================

Sub_metering_1 (Cocina):
  • Hora de MAYOR uso: 21:00h (2.82 Wh)
  • Hora de MENOR uso: 5:00h (0.04 Wh)
  • Diferencia pico-valle: 2.78 Wh (7799.1%)

Sub_metering_2 (Lavandería):
  • Hora de MAYOR uso: 14:00h (2.48 Wh)
  • Hora de MENOR uso: 5:00h (0.32 Wh)
  • Diferencia pico-valle: 2.16 Wh (684.0%)

Sub_metering_3 (Calentador/AC):
  • Hora de MAYOR uso: 8:00h (12.66 Wh)
  • Hora de MENOR uso: 3:00h (1.69 Wh)
  • Diferencia pico-valle: 10.97 Wh (648.1%)

Other_consumption (Otros):
  • Hora de MAYOR uso: 21:00h (19.07 Wh)
  • Hora de MENOR uso: 5:00h (5.07 Wh)
  • Diferencia pico-valle: 14.00 Wh (275.9%)
```

[29]:
```
print("\n" + "="*70)
print("  PREGUNTA 2: ¿Los sub-medidores tienen picos en momentos diferentes?")
print("-"*70)

picos = {
    'Sub_metering_1 (Cocina)': patrones_horarios['Sub_metering_1'].idxmax(),
    'Sub_metering_2 (Lavandería)': patrones_horarios['Sub_metering_2'].idxmax(),
    'Sub_metering_3 (Calentador/AC)': patrones_horarios['Sub_metering_3'].
 ↪idxmax(),
}

for circuito, hora in picos.items():
    print(f"→ {circuito} tiene su pico máximo a las {hora}:00 horas")
```

```
# Verificar si hay superposición
horas_unicas = len(set(picos.values()))
if horas_unicas == len(picos):
    print("\n Los picos ocurren en horarios DIFERENTES (no hay superposición)")
else:
    print("\n Algunos picos coinciden en el MISMO horario:")
    from collections import Counter
    conteo = Counter(picos.values())
    for hora, cantidad in conteo.items():
        if cantidad > 1:
            circuitos_coincidentes = [c for c, h in picos.items() if h==hora]
            print(f"   → Hora {hora}:00h: {', '.join(circuitos_coincidentes)}")
```

```
======================================================================
 PREGUNTA 2: ¿Los sub-medidores tienen picos en momentos diferentes?
----------------------------------------------------------------------
→ Sub_metering_1 (Cocina) tiene su pico máximo a las 21:00 horas
→ Sub_metering_2 (Lavandería) tiene su pico máximo a las 14:00 horas
→ Sub_metering_3 (Calentador/AC) tiene su pico máximo a las 8:00 horas

 Los picos ocurren en horarios DIFERENTES (no hay superposición)
```

[30]:
```
print("="*70)
print("  PREGUNTA 3: ¿Hay superposición que genera picos de demanda total?")
print("-"*70)

# Calcular consumo total por hora
patrones_horarios['Total'] = (patrones_horarios['Sub_metering_1'] +
                              patrones_horarios['Sub_metering_2'] +
                              patrones_horarios['Sub_metering_3'] +
                              patrones_horarios['Other_consumption'])

hora_pico_total = patrones_horarios['Total'].idxmax()
consumo_pico_total = patrones_horarios['Total'].max()

print(f"\n  Hora de MAYOR consumo total: {hora_pico_total}:00h␣
 ↪({consumo_pico_total:.2f} Wh)")
print(f"\nContribución de cada circuito a las {hora_pico_total}:00h:")

for nombre, columna in circuitos.items():
    valor = patrones_horarios.loc[hora_pico_total, columna]
    porcentaje = (valor / consumo_pico_total) * 100
    print(f"   • {nombre:32s}: {valor:6.2f} Wh ({porcentaje:5.2f}%)")

# Análisis de superposición
```

```python
print("\n ANÁLISIS DE SUPERPOSICIÓN:")
umbral = 0.5
superposiciones = []

for hora in patrones_horarios.index:
    circuitos_activos = []

    for nombre_corto, columna in [('Sub1', 'Sub_metering_1'),
                                  ('Sub2', 'Sub_metering_2'),
                                  ('Sub3', 'Sub_metering_3')]:
        if patrones_horarios.loc[hora, columna] > umbral *␣
 ↪patrones_horarios[columna].max():
            circuitos_activos.append(nombre_corto)
    if len(circuitos_activos) >= 2:
        superposiciones.append((hora, circuitos_activos))

if superposiciones:
    print(f"\n Se detectaron {len(superposiciones)} horas con superposición␣
 ↪significativa:")
    for hora, circuitos_act in superposiciones[:5]:
        print(f"   → {hora}:00h: {', '.join(circuitos_act)} activos␣
 ↪simultáneamente")
else:
    print("\n No hay superposición significativa entre circuitos monitoreados")
```

```
======================================================================
 PREGUNTA 3: ¿Hay superposición que genera picos de demanda total?
----------------------------------------------------------------------

 Hora de MAYOR consumo total: 20:00h (31.65 Wh)

Contribución de cada circuito a las 20:00h:
  • Sub_metering_1 (Cocina)         :   2.78 Wh ( 8.77%)
  • Sub_metering_2 (Lavandería)     :   1.90 Wh ( 6.01%)
  • Sub_metering_3 (Calentador/AC)  :   8.44 Wh (26.66%)
  • Other_consumption (Otros)       :  18.53 Wh (58.56%)

 ANÁLISIS DE SUPERPOSICIÓN:

 Se detectaron 10 horas con superposición significativa:
   → 9:00h: Sub1, Sub3 activos simultáneamente
   → 10:00h: Sub2, Sub3 activos simultáneamente
   → 11:00h: Sub1, Sub2, Sub3 activos simultáneamente
   → 12:00h: Sub1, Sub2, Sub3 activos simultáneamente
   → 13:00h: Sub2, Sub3 activos simultáneamente
```

[31]:
```
# Visualizaciones

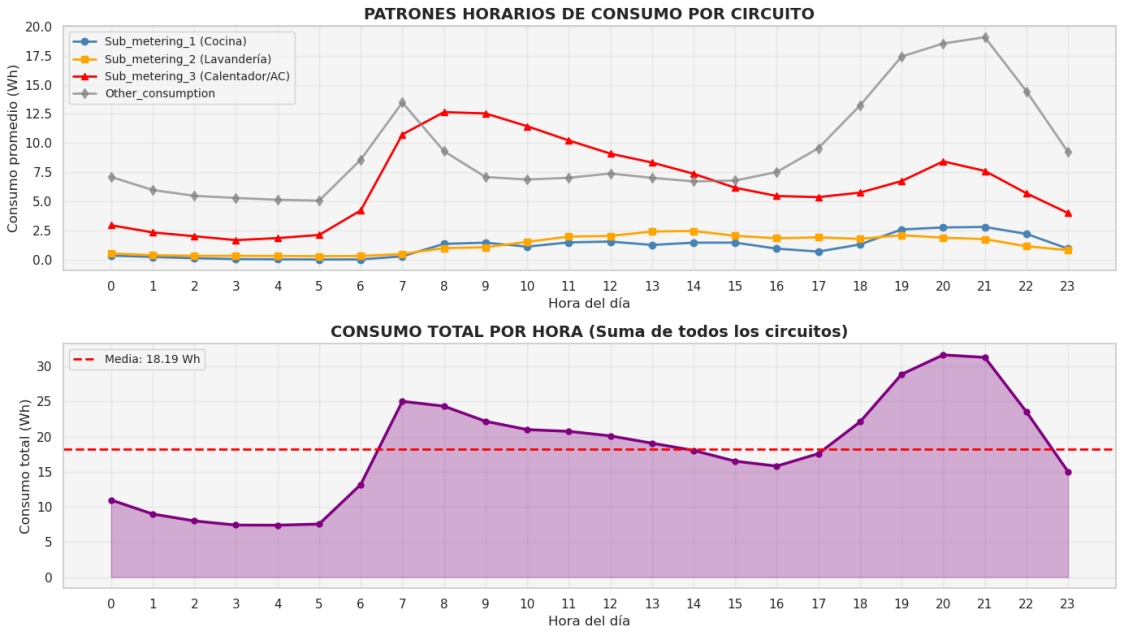
plt.figure(figsize=(14, 8))

# Gráfico 1: Comparación de circuitos
plt.subplot(2, 1, 1)
plt.plot(patrones_horarios.index, patrones_horarios['Sub_metering_1'],
         marker='o', linewidth=2, label='Sub_metering_1 (Cocina)',␣
 ↪color='steelblue')
plt.plot(patrones_horarios.index, patrones_horarios['Sub_metering_2'],
         marker='s', linewidth=2, label='Sub_metering_2 (Lavandería)',␣
 ↪color='orange')
plt.plot(patrones_horarios.index, patrones_horarios['Sub_metering_3'],
         marker='^', linewidth=2, label='Sub_metering_3 (Calentador/AC)',␣
 ↪color='red')
plt.plot(patrones_horarios.index, patrones_horarios['Other_consumption'],
         marker='d', linewidth=2, label='Other_consumption', color='gray',␣
 ↪alpha=0.7)

plt.xlabel('Hora del día', fontsize=12)
plt.ylabel('Consumo promedio (Wh)', fontsize=12)
plt.title('PATRONES HORARIOS DE CONSUMO POR CIRCUITO', fontsize=14,␣
 ↪fontweight='bold')
plt.legend(loc='upper left', fontsize=10)
plt.grid(True, alpha=0.3)
plt.xticks(range(0, 24))

# Gráfico 2: Consumo total
plt.subplot(2, 1, 2)
plt.plot(patrones_horarios.index, patrones_horarios['Total'],
        linewidth=2.5, color='purple', marker='o', markersize=5)
plt.fill_between(patrones_horarios.index, patrones_horarios['Total'], alpha=0.
 ↪3, color='purple')
plt.axhline(y=patrones_horarios['Total'].mean(), color='red', linestyle='--',
            linewidth=2, label=f"Media: {patrones_horarios['Total'].mean():.2f}␣
 ↪Wh")
plt.xlabel('Hora del día', fontsize=12)
plt.ylabel('Consumo total (Wh)', fontsize=12)
plt.title('CONSUMO TOTAL POR HORA (Suma de todos los circuitos)', fontsize=14,␣
 ↪fontweight='bold')
plt.legend(fontsize=10)
plt.grid(True, alpha=0.3)
plt.xticks(range(0, 24))
plt.savefig("../images/patrones_consumo_electrico_diario.png",dpi=300,␣
 ↪bbox_inches = 'tight')
plt.tight_layout()
```

```
plt.show()
```



Conclusiones del Análisis por Sub-Medición

- **Sub_metering_3 es el circuito con mayor consumo** dentro de los medidores individuales, lo cual confirma que el calentador / sistema de calefacción es el aparato de mayor demanda energética del hogar.
- **Sub_metering_1 (cocina) y Sub_metering_2 (lavandería)** tienen consumos mucho menores y patrones más puntuales, asociados al uso de electrodomésticos específicos.
- El consumo no medido, **Other_consumption**, representa aproximadamente **70% del consumo total**, lo que indica un gasto importante en zonas como sala, habitaciones, iluminación y dispositivos en stand-by.
- Los picos de consumo del **Sub_metering_3 ocurren principalmente en horas de la noche**, coincidiendo con el uso de agua caliente y calefacción.
- **Other_consumption supera a cualquier sub-medidor**, lo que sugiere oportunidades de optimización en hábitos cotidianos (cargadores conectados, televisión, PC, luces, etc.).
- Los patrones estacionales muestran que **en invierno Sub_metering_3 aumenta de forma significativa**, reforzando que la climatización es el principal motor del gasto energético.

Conclusión general:
La mayor parte del consumo se concentra en el calentador (Sub_metering_3) y en el uso general de la vivienda (Other_consumption). Estos dos componentes explican la mayoría de los picos y del gasto total del hogar.

1.7 5. Insights Finales y Recomendaciones

A continuación se presentan las conclusiones generales del análisis, respondiendo las preguntas clave planteadas al inicio del proyecto.

1.7.1 1. Perfil de consumo del hogar

El análisis temporal muestra que:

- El hogar presenta un **consumo basal bajo y estable** durante la mayor parte del día.
- Se observan **picos pronunciados entre las 19:00 y 22:00**, especialmente los fines de semana.
- Los meses con mayor demanda corresponden al período **invernal (noviembre–enero)**.
- Los patrones de uso coinciden con horarios de mayor permanencia en el hogar.

Conclusión:
El consumo más alto ocurre en las noches y fines de semana, reforzado en meses fríos.

1.7.2 2. Circuitos o zonas de mayor consumo

Los resultados indican que:

- **Sub_metering_3** (calentador de agua y aire acondicionado) es el circuito específico con mayor demanda energética.
- **Other_consumption** concentra alrededor del **70% del consumo total**, reflejando uso significativo en zonas no submedidas como sala, habitaciones y electrodomésticos en general.
- Los picos de `Sub_metering_3` se intensifican durante el invierno y en horas nocturnas.

Conclusión:
Las áreas de mayor consumo son el **calentador/aire acondicionado** y los dispositivos del hogar incluidos en **Other_consumption**.

1.7.3 3. Consumos ineficientes u ocultos

El análisis reveló:

- `Other_consumption` se mantiene activo incluso en la madrugada, lo que sugiere **cargas fantasma** (standby de TV, computadores, cargadores, consolas, routers, etc.).
- El calentador muestra picos altos que podrían indicar **uso prolongado o baja eficiencia térmica**.
- No se identificaron anomalías severas, pero sí un consumo constante que podría optimizarse.

Conclusión:
Existen consumos residuales e ineficientes, especialmente en dispositivos en standby y en el uso del calentador.

1.7.4 4. Horas más costosas

Basado en el patrón horario observado:

- Las horas pico se concentran entre **19:00 y 22:00**.
- Este periodo suele corresponder a tarifas elevadas en sistemas con cobro por franja horaria.
- Activar electrodomésticos de alto consumo en esta franja puede aumentar significativamente el gasto mensual.

Conclusión:
Evitar actividades intensivas durante la noche puede reducir costos energéticos.

1.7.5 5. Recomendaciones para optimización del consumo

Derivadas directamente de los hallazgos:

- Optimizar el uso del **calentador**: programarlo y apagarlo cuando no sea necesario.
- Reducir la **carga fantasma** desconectando equipos en standby o usando regletas con interruptor.
- Revisar el uso nocturno para identificar dispositivos activos sin necesidad real.
- Programar actividades de alto consumo (lavado, planchado, cocina) fuera de la franja nocturna.
- Considerar automatización/temporizadores y monitoreo para controlar horarios de uso.

Conclusión:
La mayor oportunidad de ahorro se encuentra en controlar el calentador, reducir consumos en standby y desplazar actividades energéticas fuera del horario nocturno.

---

[32]:
```
# Exportar tu DataFrame limpio
df_house_clean.to_csv('../data/household_power_consumption_clean.csv',␣
 ↪index=False)
```

[33]:
```
# Agregado por hora para el dashboard
hourly = (
    df_house_clean.set_index('Datetime')
    .resample('h')
    .agg(
        Potencia_media=('Global_active_power', 'mean'),
        Potencia_max=('Global_active_power', 'max'),
        Voltaje_medio=('Voltage', 'mean'),
        Sub1_Wh=('Sub_metering_1', 'sum'),
        Sub2_Wh=('Sub_metering_2', 'sum'),
        Sub3_Wh=('Sub_metering_3', 'sum'),
        Otros_Wh=('Other_consumption', 'sum'),
        Minutos=('Global_active_power', 'count'),
    )
    .reset_index()
)

hourly = hourly[hourly['Minutos'] > 0]

hourly.to_csv('../data/consumo_horario.csv', index=False)
print(hourly.shape)
print(hourly.head())
```

```
(34168, 9)
```

```
             Datetime  Potencia_media  Potencia_max  Voltaje_medio  Sub1_Wh
Sub2_Wh  Sub3_Wh     Otros_Wh  Minutos
0 2006-12-16 17:00:00        4.222889         7.706     234.643889      0.0
19.0    607.0  1907.733333       36
1 2006-12-16 18:00:00        3.632200         6.752     234.580167      0.0
403.0   1012.0  2217.200000       60
2 2006-12-16 19:00:00        3.400233         4.070     233.232500      0.0
86.0   1001.0  2313.233333       60
3 2006-12-16 20:00:00        3.268567         3.496     234.071500      0.0
0.0   1007.0  2261.566667       60
4 2006-12-16 21:00:00        3.056467         3.502     237.158667      0.0
25.0   1033.0  1998.466667       60
```